JavaScript is disabled on your browser.

compact1, compact2, compact3

java.nio.file.attribute

Interface AclFileAttributeView

All Superinterfaces:

`AttributeView`, `FileAttributeView`, `FileOwnerAttributeView`

```
public interface AclFileAttributeView
extends FileOwnerAttributeView
```

A file attribute view that supports reading or updating a file's Access Control Lists (ACL) or file owner attributes.

ACLs are used to specify access rights to file system objects. An ACL is an ordered list of `access-control-entries`, each specifying a `UserPrincipal` and the level of access for that user principal. This file attribute view defines the `getAcl`, and `setAcl` methods to read and write ACLs based on the ACL model specified in *RFC 3530: Network File System (NFS) version 4 Protocol*. This file attribute view is intended for file system implementations that support the NFSv4 ACL model or have a *well-defined* mapping between the NFSv4 ACL model and the ACL model used by the file system. The details of such mapping are implementation dependent and are therefore unspecified.

This class also extends `FileOwnerAttributeView` so as to define methods to get and set the file owner.

When a file system provides access to a set of `file-systems` that are not homogeneous then only some of the file systems may support ACLs. The `supportsFileAttributeView` method can be used to test if a file system supports ACLs.

Interoperability

RFC 3530 allows for special user identities to be used on platforms that support the POSIX defined access permissions. The special user identities are `"OWNER@"`,

"GROUP@", and "EVERYONE@". When both the AclFileAttributeView and the PosixFileAttributeView are supported then these special user identities may be included in ACL entries that are read or written. The file system's UserPrincipalLookupService may be used to obtain a UserPrincipal to represent these special identities by invoking the lookupPrincipalByName method.

Usage Example: Suppose we wish to add an entry to an existing ACL to grant "joe" access:

```
// lookup "joe"
UserPrincipal joe = file.getFileSystem().getUserPrincipalLookupService()
    .lookupPrincipalByName("joe");

// get view
AclFileAttributeView view = Files.getFileAttributeView(file, AclFileAttributeV

// create ACE to give "joe" read access
AclEntry entry = AclEntry.newBuilder()
    .setType(AclEntryType.ALLOW)
    .setPrincipal(joe)
    .setPermissions(AclEntryPermission.READ_DATA, AclEntryPermission.READ_ATTR
    .build();

// read ACL, insert ACE, re-write ACL
List<AclEntry> acl = view.getAcl();
acl.add(0, entry);   // insert before any DENY entries
view.setAcl(acl);
```

Dynamic Access

Where dynamic access to file attributes is required, the attributes supported by this attribute view are as follows:

Name	Type
"acl"	List<AclEntry>
"owner"	UserPrincipal

The getAttribute method may be used to read the ACL or owner attributes as if by invoking the getAcl or getOwner methods.

The setAttribute method may be used to update the ACL or owner attributes as if by invoking the setAcl or setOwner methods.

Setting the ACL when creating a file

Implementations supporting this attribute view may also support setting the initial ACL when creating a file or directory. The initial ACL may be provided to methods

such as `createFile` or `createDirectory` as an `FileAttribute` with `name` `"acl:acl"` and a `value` that is the list of `AclEntry` objects.

Where an implementation supports an ACL model that differs from the NFSv4 defined ACL model then setting the initial ACL when creating the file must translate the ACL to the model supported by the file system. Methods that create a file should reject (by throwing `IOException`) any attempt to create a file that would be less secure as a result of the translation.

Since:

1.7

Method Summary

Abstract Methods

Modifier and Type	Method	Description
`List<AclEntry>`	`getAcl()`	Reads the access control list.
`String`	`name()`	Returns the name of the attribute view.
`void`	`setAcl(List<AclEntry> acl)`	Updates (replace) the access control list.

Methods inherited from interface java.nio.file.attribute.FileOwnerAttributeView

`getOwner`, `setOwner`

Method Detail

name

`String name()`

Returns the name of the attribute view. Attribute views of this type have the name `"acl"`.

Specified by:

`name` in interface `AttributeView`

Specified by:

`name` in interface `FileOwnerAttributeView`

Returns:

the name of the attribute view

getAcl

```
List<AclEntry> getAcl()
               throws IOException
```

Reads the access control list.

When the file system uses an ACL model that differs from the NFSv4 defined ACL model, then this method returns an ACL that is the translation of the ACL to the NFSv4 ACL model.

The returned list is modifiable so as to facilitate changes to the existing ACL. The `setAcl` method is used to update the file's ACL attribute.

Returns:

an ordered list of `entries` representing the ACL

Throws:

`IOException` - if an I/O error occurs

`SecurityException` - In the case of the default provider, a security manager is installed, and it denies `RuntimePermission("accessUserInformation")` or its `checkRead` method denies read access to the file.

setAcl

```
void setAcl(List<AclEntry> acl)
     throws IOException
```

Updates (replace) the access control list.

Where the file system supports Access Control Lists, and it uses an ACL model that differs from the NFSv4 defined ACL model, then this method must translate the ACL to the model supported by the file system. This method should reject (by throwing `IOException`) any attempt to write an ACL that would appear to make the file more secure than would be the case if the ACL were updated. Where an implementation does not support a mapping of `AclEntryType.AUDIT` or `AclEntryType.ALARM` entries, then this method ignores these entries when writing the ACL.

If an ACL entry contains a `user-principal` that is not associated with the same provider as this attribute view then `ProviderMismatchException` is thrown. Additional validation, if any, is implementation dependent.

If the file system supports other security related file attributes (such as a file `access-permissions` for example), the updating the access control list may also cause these security related attributes to be updated.

Parameters:

`acl` - the new access control list

Throws:

`IOException` - if an I/O error occurs or the ACL is invalid

`SecurityException` - In the case of the default provider, a security manager is installed, it denies `RuntimePermission("accessUserInformation")` or its `checkWrite` method denies write access to the file.

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.